OFFSIDE LABS

DAMM v2

Release 0.2.0

Smart Contract Security Assessment

March 2026

Prepared for:

Meteora

Prepared by:

Offside Labs

Sirius Xie

Contents

1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.

 **https://offside.io/**

 **https://github.com/offsidelabs**

 **https://twitter.com/offside_labs**

2 Executive Summary

Introduction

Offside Labs completed a security audit of *DAMM v2* smart contracts, starting on February 26th, 2026, and concluding on March 3th, 2026.

Project Overview

DAMM v2 Release 0.2.0 adds a compounding fee mode and a constant product pool implementation, updates swap settlement and event data accordingly, removes partner fee related fields and the partner fee claim instruction, adds a pool layout version with lazy migration during swap add and remove to stay compatible with legacy pools, adjusts base fee structures and serialization to remove extra padding while keeping the on-chain size fixed, and refactors protocol fee zapping with updated validations.

Audit Scope

The assessment scope contains mainly the smart contracts of the cp-amm program for the *DAMM v2* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- DAMM v2
  - Codebase: `https://github.com/MeteoraAg/damm-v2`
  - Release 0.2.0: `PR-187`
  - Commit Hash: 8168ac6e94bfb1940488593d14014f0c30d34aa7

The issues described in this report have been resolved by the following commit:

- DAMM v2
  - Codebase: `https://github.com/MeteoraAg/damm-v2`
  - Commit Hash: 6e9aee4e549bd792c8f5b82b88be04459e644f3c

We listed the files we have audited below:

- DAMM v2 Release 0.2.0
  - programs/cp-amm/src/base_fee/fee_market_cap_scheduler.rs
  - programs/cp-amm/src/base_fee/fee_rate_limiter.rs
  - programs/cp-amm/src/base_fee/fee_time_scheduler.rs
  - programs/cp-amm/src/constants.rs
  - programs/cp-amm/src/error.rs
  - programs/cp-amm/src/event.rs
  - programs/cp-amm/src/instructions/initialize_pool/ix_initialize_customizable_pool.rs
  - programs/cp-amm/src/instructions/initialize_pool/ix_initialize_pool.rs

- programs/cp-amm/src/instructions/initialize_pool/ix_initialize_pool_with_dynamic_config.rs
- programs/cp-amm/src/instructions/ix_add_liquidity.rs
- programs/cp-amm/src/instructions/ix_remove_liquidity.rs
- programs/cp-amm/src/instructions/mod.rs
- programs/cp-amm/src/instructions/operator/ix_create_static_config.rs
- programs/cp-amm/src/instructions/operator/ix_fix_pool_layout_version.rs
- programs/cp-amm/src/instructions/operator/ix_zap_protocol_fee.rs
- programs/cp-amm/src/instructions/operator/mod.rs
- programs/cp-amm/src/instructions/swap/ix_p_swap.rs
- programs/cp-amm/src/lib.rs
- programs/cp-amm/src/liquidity_handler/compounding_liquidity.rs
- programs/cp-amm/src/liquidity_handler/concentrated_liquidity.rs
- programs/cp-amm/src/liquidity_handler/mod.rs
- programs/cp-amm/src/math/safe_math.rs
- programs/cp-amm/src/params/fee_parameters.rs
- programs/cp-amm/src/state/config.rs
- programs/cp-amm/src/state/fee.rs
- programs/cp-amm/src/state/operator.rs
- programs/cp-amm/src/state/pool.rs

Findings

The security audit revealed:

- 0 critical issue
- 0 high issue
- 0 medium issue
- 1 low issues
- 0 informational issue

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.

3 Summary of Findings

ID	Title	Severity	Status
01	Stale pool.sqrt_price After Add/Remove Liquidity in Compounding Mode Pool	Low	Acknowledged

4 Key Findings and Recommendations

4.1 Stale pool.sqrt_price After Add/Remove Liquidity in Compounding Mode Pool

Severity: Low	Status: Acknowledged
Target: Smart Contract	Category: Precision

Description

When `pool.collect_fee_mode` is `CollectFeeMode::Compounding`, the pool's real price is determined by `token_b_amount / token_a_amount`. Currently, `pool.sqrt_price` is recalculated and updated only during swap. In `add_liquidity` and `remove_liquidity` IXs, the program updates `pool.liquidity`, `pool.token_a_amount`, and `pool.token_b_amount`, but does not update `pool.sqrt_price`.

In compounding pools, the token amounts required for add/remove are computed with rounding (`add` uses `rounding up`, `remove` uses `rounding down`). This can make the rounding error on token A and token B not perfectly symmetric, causing a small drift in the ratio `token_b_amount / token_a_amount`. As a result, after add/remove the pool's real price may have changed, while `pool.sqrt_price` still remains stale until the next swap updates it.

Impact

A stale `pool.sqrt_price` may affect the following components:

- When `BaseFeeMode` is `FeeMarketCapScheduler*`, `pool.sqrt_price` is used as `current_sqrt_price` to compute the base fee. Using a stale `pool.sqrt_price` may cause the effective fee tier/period to differ from what it should be given the updated reserves.
- For pools with Dynamic fee enabled, Dynamic fee updates its `reference/volatility` around swaps based on `pool.sqrt_price`. If add/remove in compounding mode causes a drift in `token_b_amount / token_a_amount` but `pool.sqrt_price` is not updated accordingly, that price drift can be underestimated during the interval; when a subsequent swap finally updates `sqrt_price`, the drift may be accounted for all at once, potentially increasing `variable_fee` for subsequent trades and keeping it elevated for some time.

Recommendation

When `pool.collect_fee_mode` is `CollectFeeMode::Compounding`, recompute and write back `pool.sqrt_price` after `add/remove_liquidity` updates `token_a_amount/token_b_amount`, so that `pool.sqrt_price` always reflects the latest price implied by current reserves.

Mitigation Review Log

Meteora Team: We ACK this, since sqrt price will affect on dynamic fee and fee by market cap base fee.

5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

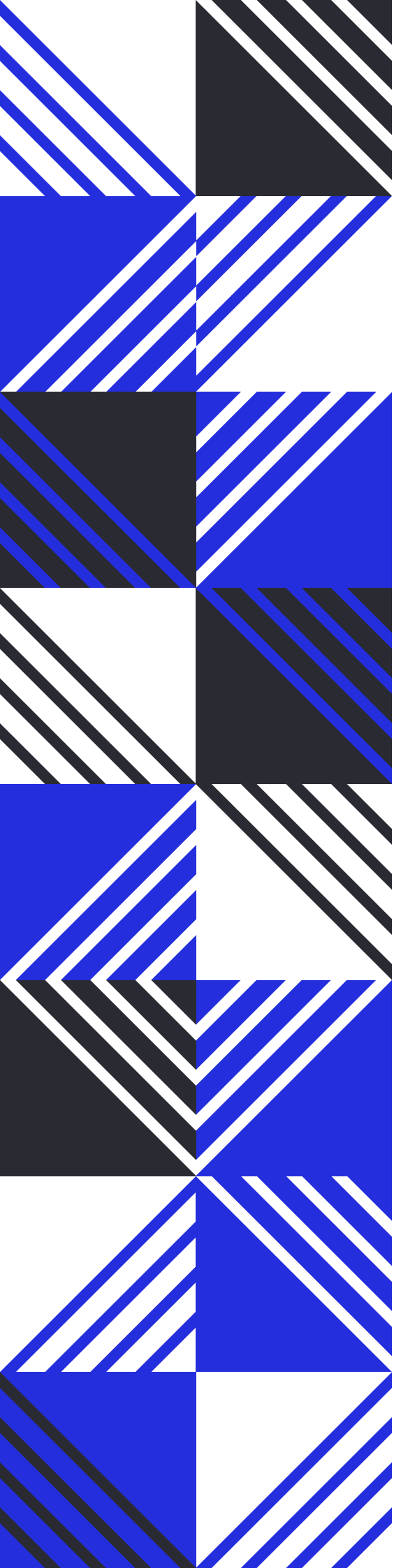
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



OFFSIDE LABS

https://offside.io/

https://github.com/offsidelabs

https://twitter.com/offside_labs